План переработки входных и выходных параметров программы

Frontend + Backend API contract для проекта “AI Directional Prediction of Thermoelastic Waves in Geological Media”

Дата анализа: 17 мая 2026. Репозиторий: github.com/askarov32/ai-directional-prediction. Документ основан на текущей структуре backend/frontend, model-service contract и методологии дипломной работы.

1. Краткий вывод

Текущий контракт уже имеет правильную архитектурную идею: frontend отправляет единый prediction request, backend валидирует его, обогащает параметрами материала, маршрутизирует в выбранный model service и нормализует ответ. Однако текущий response слишком “direction-demo oriented”: он возвращает direction_vector, azimuth, elevation, wave_type, travel_time, magnitude, max_displacement и max_temperature_perturbation. Для дипломной постановки этого недостаточно, потому что явно не возвращаются predicted temperature, temperature perturbation as point value, displacement components u/v, displacement magnitude, source-probe distance и fallback status.

Рекомендация: сделать новый normalized contract v2 для 2D-прототипа. Он должен быть компактным, физически интерпретируемым и одинаковым для PINN, FNO, MeshGraphNet и Transformer-style routes. Старый контракт можно поддержать временно как compatibility layer, но frontend лучше перевести на v2-поля.

2. Что сейчас есть в проекте

Компонент	Текущее состояние	Проблема для дипломного MVP
Backend	FastAPI orchestrator: принимает unified request, resolve medium, route to model service, normalize response.	Это хорошая основа. Нужно изменить не архитектуру, а schema + normalizer + payload contract.
Request	model, medium_id, scenario.temperature_c, pressure_mpa, time_ms, source, probe, domain.	Есть лишний 3D-first шум для дипломного 2D-прототипа. Нет явного thermal_state: T_ref, T_source, theta.
Response	prediction.direction_vector, azimuth_deg, elevation_deg, magnitude, wave_type, travel_time_ms; field_summary.max_displacement, max_temperature_perturbation; meta.model_version, latency, request_id.	Нет явных T, theta, u, v. magnitude неоднозначен: неясно, это displacement magnitude или comparative score.
Frontend	Форма собирает 3D/2D request, UI показывает direction, azimuth, elevation, magnitude, travel time, max displacement, perturbation.	UI не показывает главные физические outputs как единый thermoelastic response.
Model services	FNO/PINN/MGN/mock возвращают или flat response, или старую normalized shape.	Нужно единое v2-ядро: direct predictions + derived geometry + metadata.

3. Целевой принцип контракта

- Frontend вводит только физически понятные и визуально объяснимые параметры: material, thermal_state, source, probe, observation time, simplified domain/scenario.
- Backend сам выводит derived quantities: theta, distance, propagation vector, unit direction, azimuth, material derived constants.
- Model service по возможности возвращает direct predictions: temperature or perturbation, u, v. Backend нормализует их в единую структуру.
- Stress и strain не делать обязательными public outputs. Они допускаются только как optional derived diagnostics, если есть поле/градиенты displacement.

- Для диплома основной режим должен быть rect_2d. 3D-поля оставить как future extension, чтобы не перегружать frontend и защиту.
- Metadata отделить от физики: latency, model_version, fallback_used, fallback_reason, warnings, request_id.

4. Новый входной контракт v2

4.1 Минимальный request для frontend

Это основной формат, который должен собирать frontend. Материал выбирается из backend catalog, а его физические свойства backend подставляет сам.


```
{
  "schema_version": "2.0",
  "model": "pinn",
  "medium_id": "sandstone_medium",
  "thermal_state": {
    "reference_temperature_k": 293.15,
    "source_temperature_k": 350.0
  },
  "geometry": {
    "dimension": 2,
    "source": {
      "x_m": 0.0,
      "y_m": 0.0
    },
    "probe": {
      "x_m": 1.0,
      "y_m": 0.5
    }
  },
  "observation": {
    "time_s": 0.1
  },
  "scenario": {
    "thermal_source_type": "point",
    "mechanical_constraint": "free",
    "boundary_condition_type": "prototype_simplified"
  }
}
```


4.2 Полный request для backend/model-service

Backend после resolve medium и валидации может сформировать enriched payload. Его можно отправлять во все model services одинаково.


```
{
  "schema_version": "2.0",
  "model": {
    "name": "fno",
    "allow_fallback": true
  },
  "material": {
    "id": "sandstone_medium",
    "name": "Sandstone",
    "category": "sedimentary",
    "properties": {
      "rho_kg_m3": 2200.0,
      "young_modulus_pa": null,
      "poisson_ratio": null,
      "vp_m_s": 3000.0,
      "vs_m_s": 1700.0,
      "thermal_conductivity_w_mk": 2.5,
      "heat_capacity_j_kgk": 800.0,
      "thermal_expansion_1_k": 1e-05
    },
    "derived_properties": {
      "shear_modulus_pa": "computed if possible",
      "bulk_modulus_pa": "computed if possible",
      "lame_lambda_pa": "computed if possible",
      "volumetric_heat_capacity_j_m3k": "rho * Cp"
    }
  },
  "thermal_state": {
    "reference_temperature_k": 293.15,
    "source_temperature_k": 350.0,
    "temperature_perturbation_k": 56.85
  },
  "geometry": {
    "dimension": 2,
    "source": {
      "x_m": 0.0,
      "y_m": 0.0
    },
```

```
    "probe": {
      "x_m": 1.0,
      "y_m": 0.5
    },
    "derived": {
      "propagation_vector_m": {
        "dx": 1.0,
        "dy": 0.5
      },
      "distance_m": 1.118,
      "unit_direction": {
        "x": 0.894,
        "y": 0.447
      },
      "azimuth_deg": 26.565
    }
  },
  "observation": {
    "time_s": 0.1
  },
  "model_runtime": {
    "representation": "grid",
    "requested_outputs": [
      "temperature",
      "displacement",
      "direction"
    ]
  }
}
```

4.3 Required / optional / derived inputs

Группа	Required	Optional	Derived by backend
Model	model.name	allow_fallback	representation, routing_hint
Material	medium_id или material preset	manual material override	rho/Cp-based heat capacity, elastic constants
Thermal state	reference_temperature_k + source_temperature_k или temperature_perturbation_k	heat_flux, source_radius	theta = T_source - T_ref
Geometry	source.x/y, probe.x/y, dimension=2	source direction as comparison vector	distance, unit direction, azimuth
Observation	time_s	time array for batch prediction	time_ms compatibility conversion
Domain/scenario	prototype defaults	boundary labels, resolution	rect_2d default domain if omitted

5. Новый выходной контракт v2

Главное изменение: prediction должен быть разделен на thermal_response, displacement, directional_response, optional_outputs и metadata. Тогда frontend сможет показывать физику без путаницы.

```
{
  "schema_version": "2.0",
  "request_id": "uuid",
  "status": "ok",
  "model": {
    "name": "pinn",
    "version": "pinn-baseline-v1",
    "route": "/predict",
    "inference_time_ms": 34.2,
    "fallback_used": false,
    "fallback_reason": null
  },
  "material": {
    "id": "sandstone_medium",
    "name": "Sandstone",
    "category": "sedimentary"
  },
  "geometry": {
    "dimension": 2,
    "source": {
      "x_m": 0.0,
      "y_m": 0.0
    },
    "probe": {
      "x_m": 1.0,
      "y_m": 0.5
    },
    "propagation_vector_m": {
      "dx": 1.0,
      "dy": 0.5
    },
```

```
    "unit_direction": {
      "x": 0.89443,
      "y": 0.44721
    },
    "distance_m": 1.11803,
    "azimuth_deg": 26.565,
    "azimuth_convention": "atan2(dy, dx), degrees, xy-plane"
  },
  "prediction": {
    "thermal": {
      "temperature_k": {
        "value": 315.2,
        "source": "direct_model_prediction"
      },
      "temperature_perturbation_k": {
        "value": 22.05,
        "reference_temperature_k": 293.15,
        "source": "derived_from_temperature"
      }
    },
    "displacement": {
      "components_m": {
        "u": 1.2e-06,
        "v": 3e-07
      },
      "magnitude_m": 1.237e-06,
      "components_source": "direct_model_prediction",
      "magnitude_source": "derived_from_u_v"
    },
    "directional_response": {
      "distance_m": 1.11803,
      "azimuth_deg": 26.565,
      "response_magnitude_score": 0.73
    }
  },
  "optional_outputs": {
    "confidence_score": null,
    "travel_time_s": null,
    "field_summary": {
      "max_displacement_m": null,
      "max_temperature_perturbation_k": null
    },
    "strain": null,
    "stress": null,
    "field_grid": null
  },
  "diagnostics": {
    "warnings": [],
    "notes": [
      "Prototype prediction; not a field-validated thermoelastic simulation."
    ]
  }
}
```


5.1 Классификация outputs

Категория	Поля	Кто считает
Required physical outputs	temperature_k, temperature_perturbation_k, displacement.u, displacement.v, displacement_magnitude_m	model + backend normalization
Required directional outputs	propagation vector, unit direction, distance_m, azimuth_deg	backend from source/probe
Optional outputs	confidence_score, travel_time_s, field_grid, max field summaries	model or post-processing
Derived diagnostics	strain, stress, velocity, acceleration	backend/service only if enough field data
Model-comparison metadata	model name/version, latency, fallback_used, warnings, request_id	backend + service runtime

6. Backend implementation plan

6.1 Files to change

Файл	Что изменить	Цель
backend/app/schemas/prediction.py	Добавить PredictionRequestV2Schema, ThermalStateSchema, Geometry2DSchema, ObservationSchema, ScenarioPrototypeSchema, PredictionResponseV2Schema.	Официальный API-контракт.
backend/app/domain/entities/prediction.py	Добавить dataclass: ThermalState, Geometry2D, Observation, DerivedGeometry, NormalizedPredictionOutput.	Чистая domain-модель без frontend-шумов.

Файл	Что изменить	Цель
backend/app/domain/services/derived_quantities.py	Новый файл: compute_theta, compute_distance, compute_unit_direction, compute_azimuth, compute_displacement_magnitude, derive_elastic_constants.	Один источник формул.
backend/app/domain/use_cases/predict_direction.py	Перед route call собрать enriched v2 payload: medium + thermal + geometry + derived values.	Не заставлять model services считать одно и то же по-разному.
backend/app/infrastructure/adapters/remote_response_schema.py	Поддержать v2 remote payload + old flat payload. Старый формат маппить в optional/legacy.	Мягкая миграция без одномоментного полома всех сервисов.
backend/app/infrastructure/adapters/response_normalizer.py	Вернуть response_v2. Добавить fallback_used, fallback_reason, diagnostics, units.	Frontend получает физически понятный output.
backend/app/infrastructure/clients/*.py	В build_payload добавлять schema_version=2.0, requested_outputs=[temperature, displacement, direction], representation.	Единый remote contract.
backend/data/media/catalog.json	Добавить young_modulus_pa и poisson_ratio или явно документировать derivation from vp/vs/rho.	Материальные параметры связать с thesis formulation.

6.2 Derived formulas to centralize

```
temperature_perturbation_k = source_temperature_k - reference_temperature_k
propagation_vector = probe - source
distance_m = sqrt(dx^2 + dy^2)
unit_direction = propagation_vector / distance_m
azimuth_deg = atan2(dy, dx) * 180 / pi
displacement_magnitude_m = sqrt(u^2 + v^2)
volumetric_heat_capacity = rho * Cp
if E and nu are known:
  shear_modulus = E / (2 * (1 + nu))
  bulk_modulus = E / (3 * (1 - 2*nu))
  lame_lambda = E*nu / ((1 + nu) * (1 - 2*nu))
if vp, vs and rho are known:
  shear_modulus = rho * vs^2
  bulk_modulus = rho * (vp^2 - 4*vs^2/3)
```

6.3 Backward compatibility

- Сохранить POST /api/v1/predictions, но принимать schema_version. Если schema_version отсутствует, считать request старым v1.
- Временно отдавать оба блока: prediction_v2 как основной и legacy_prediction как optional, либо добавить query flag ?contract=v2.
- RemoteResponseSchema должен принимать старый flat response от mock/MGN/FNO и конвертировать его: direction_vector -> geometry/directional_response, max_displacement -> optional_outputs.field_summary.
- После переноса frontend удалить визуальную зависимость от elevation_deg и wave_type как обязательных полей.

7. Model service implementation plan

Сервис	Что должен принимать	Что должен возвращать
PINN	schema_version=2.0, material, thermal_state, geometry, observation, requested_outputs.	temperature_k или theta_k, u_m, v_m, optional physics_residuals. Stress/strain только diagnostics.
FNO	Тот же v2 payload + representation=grid. Для текущего MVP сохранить rect_2d/Z=1.	probe-level sampled temperature/u/v + optional field_summary. field_grid только если безопасно по размеру.
MeshGraphNet	Тот же v2 payload + representation=graph.	Если rollout есть - summary; если fallback - deterministic v2 response with fallback_used=true.
Transformer/mock	Тот же v2 payload + representation=sequence/mock.	Детерминированный demo response, но явно помеченный как fallback/demo.

7.1 Remote model response shape

```
{
  "schema_version": "2.0",
  "model_version": "service-specific-version",
  "prediction_raw": {
```

```
    "temperature_k": 315.2,
    "temperature_perturbation_k": null,
    "displacement_m": {
      "u": 1.2e-06,
      "v": 3e-07
    }
  },
  "optional_outputs": {
    "confidence_score": null,
    "travel_time_s": null,
    "field_summary": {
      "max_displacement_m": 1.5e-06,
      "max_temperature_perturbation_k": 25.0
    }
  },
  "diagnostics": {
    "fallback_used": false,
    "fallback_reason": null,
    "warnings": []
  }
}
```


8. Frontend implementation plan

8.1 Files to change

Файл	Что изменить	Результат
frontend/index.html	Переименовать sections: Thermal input, Geometry, Observation, Simplified scenario. Убрать 3D как default. Добавить result cards для T, theta, u, v, |u|, distance, azimuth.	UI станет ближе к thesis language.
frontend/assets/scripts/form.js	DEMO_TEMPLATE заменить на schema_version=2.0. readPayloadFromForm собирать thermal_state + geometry + observation. z/elevation оставить hidden/future.	Чистый 2D request.
frontend/assets/scripts/validators.js	Добавить проверки: T_ref > 0 K, source_temperature >= 0 K, source != probe, time_s > 0, finite coordinates.	Меньше некорректных запросов.
frontend/assets/scripts/ui.js	renderResult читать response.prediction.thermal, response.prediction.displacement, response.geometry, response.model. Добавить fallback badge.	Физически понятный result view.
frontend/assets/scripts/charts.js	SVG preview строить по source/probe geometry; optional heatmap только если field_grid есть.	Визуализация направления без псевдо-solver claims.
frontend/assets/scripts/state.js	Хранить contractVersion, lastDerivedGeometry, lastDiagnostics.	Debug проще.
frontend/assets/scripts/api.js	Добавить обработку schema_version и diagnostics warnings в error/debug panel.	Прозрачность результатов.

8.2 Новый result layout

- Thermal response: Predicted temperature (K), temperature perturbation (K), reference temperature.
- Mechanical response: displacement u, displacement v, displacement magnitude.
- Directional geometry: distance, unit direction vector, azimuth.
- Model comparison: model name, version, latency, fallback badge, warnings.
- Optional diagnostics: field summary, stress/strain only if available and marked as derived.

9. Implementation phases

Phase	Scope	Deliverable
0. Contract document	Create docs/api-contract-v2.md with request/response examples and units.	Документированный target contract.
1. Backend schemas	Add v2 Pydantic schemas, derived_quantities service, enriched payload.	Backend accepts v2 request.
2. Normalizer	Update remote_response_schema and response_normalizer to output v2 and map old responses.	Unified v2 response from all routes.
3. Model services	Update PINN, FNO, MGN, mock/transformer response to prediction_raw + diagnostics.	All services pass contract tests.
4. Frontend	Rewrite form builder and result renderer to v2.	UI shows T/theta/u/v/distance/azimuth/fallback.

Phase	Scope	Deliverable
5. Tests and docs	Update pytest, curl examples, README, OpenAPI screenshots.	Stable demo and thesis-consistent explanation.

10. Tests that must be added or updated

- backend/tests/test_prediction_schema.py: validates minimal v2 request, invalid source=probe, invalid temperatures, rect_2d defaults.
- backend/tests/test_response_normalizer.py: remote v2 response -> normalized response; old flat response -> normalized v2 compatibility.
- backend/tests/test_predict_direction_use_case.py: medium is resolved, thermal_state and geometry derived values are attached.
- pinn-service/tests/test_api_contract_v2.py: service returns temperature/displacement fields or explicit fallback diagnostics.
- fno-service/tests/test_api_contract_v2.py: rect_2d response ignores z, returns u/v and optional field_summary.
- frontend smoke: demo payload has schema_version=2.0 and no mandatory 3D fields.

11. Example cURL after migration

```
curl -s -X POST http://localhost:8080/api/v1/predictions \
  -H "Content-Type: application/json" \
  --data '{
    "schema_version": "2.0",
    "model": "pinn",
    "medium_id": "sandstone_medium",
    "thermal_state": {
      "reference_temperature_k": 293.15,
      "source_temperature_k": 350.0
    },
    "geometry": {
      "dimension": 2,
      "source": {"x_m": 0.0, "y_m": 0.0},
      "probe": {"x_m": 1.0, "y_m": 0.5}
    },
    "observation": {"time_s": 0.1},
    "scenario": {
      "thermal_source_type": "point",
      "mechanical_constraint": "free",
      "boundary_condition_type": "prototype_simplified"
    }
  }'
```

12. Acceptance criteria

Area	Done when
API request	Frontend sends schema_version=2.0 with material, thermal_state, geometry, observation, simplified scenario.
API response	Every model route returns T/theta/u/v, displacement magnitude, distance, azimuth, model metadata, fallback status.
Physics wording	Stress and strain are optional derived diagnostics, not mandatory direct predictions.
2D consistency	rect_2d means z=0, direction_z=0, elevation omitted or optional.
Comparison	Responses are comparable across PINN/FNO/MGN/Transformer because units and metadata are consistent.
Thesis safety	No UI or docs claim complete field-validated thermoelastic simulation.

13. Short thesis-safe wording

The practical software prototype uses a normalized two-dimensional input-output contract for AI-assisted directional prediction of thermoelastic wave behavior. The input side describes the geological material, thermal state, source-probe geometry, and observation time. The backend derives the propagation vector, distance, unit direction, azimuth, temperature perturbation, and selected material constants in order to keep these definitions consistent across model routes. The output side reports predicted temperature response, temperature perturbation, in-plane displacement components, displacement magnitude, directional geometry, and model-comparison metadata. Stress and strain are not mandatory direct outputs of every route; they may be included only as derived diagnostics when sufficient spatial information is available. This contract supports comparison between predictive model components without presenting the prototype as a full field-validated thermoelastic simulator.

14. Practical note for implementation

Do not start by rewriting the whole frontend. First freeze docs/api-contract-v2.md and backend schemas. Then make response_normalizer able to produce v2 even from old service responses. After that, migrate frontend rendering. This order reduces risk because old services can continue to run while the UI and thesis-facing contract become physically clearer.